

reduction explanation

13 May 2026 01:47

HPC Assignment 5 — Parallel Reduction Operations using OpenMP

This assignment performs:

Parallel Minimum, Maximum, Sum, and Average Calculation

using:

- OpenMP reduction
- parallel loops
- shared data processing

1. What We Have To Do?

We have to:

1. input array elements
2. find:
 - minimum value
 - maximum value
 - sum
 - average
3. perform operations in parallel using OpenMP
4. measure execution time

Main Goal

To demonstrate:

reduction operations in parallel programming

2. What is Reduction?

MOST IMPORTANT CONCEPT.

Reduction means:

combining multiple values into one final result

Examples

Operation	Final Result
-----------	--------------

Sum	one total value
-----	-----------------

Maximum	one largest value
---------	-------------------

Minimum	one smallest value
---------	--------------------

Example

Array:

2 5 1 8

Results:

Operation	Output
-----------	--------

Min	1
Max	8
Sum	16
Avg	4

Why Reduction Needed in Parallel Programming?

Suppose:

- multiple threads process different parts of array

Each thread computes:

- partial sum
- partial min
- partial max

Then:

results combined safely

This process is called:

reduction

3. Overall Workflow

Input Array



Divide Work Among Threads



Each Thread Computes Partial Results



Reduction Combines Results



Final Min/Max/Sum/Avg

4. Header Files

```
#include <iostream>
```

```
#include <vector>
```

```
#include <omp.h>
```

```
#include <climits>
```

Purpose

Header	Purpose
--------	---------

iostream	input/output
vector	dynamic array
omp.h	OpenMP
climits	INT_MAX and INT_MIN

Why climits Needed?

Provides:

- maximum integer value
- minimum integer value

Example

INT_MAX

≈ very large positive number.

INT_MIN

≈ very small negative number.

5. Input Array

```
vector<int> arr(n);
```

Stores elements.

Input Loop

```
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}
```

Reads array elements.

6. Initialize Variables

```
int min_val = INT_MAX;
```

```
int max_val = INT_MIN;
```

```
long long sum = 0;
```

Why min_val = INT_MAX ?

Because:

- any array element will be smaller

Example:

```
5 < INT_MAX
```

So minimum updates correctly.

Why max_val = INT_MIN ?

Because:

- any element will be larger

Example:

```
5 > INT_MIN
```

Why long long sum ?

To avoid:

integer overflow

for large sums.

7. Start Timer

```
double start = omp_get_wtime();
```

Records execution start time.

8. MOST IMPORTANT PART — Reduction Clause

Code:

```
#pragma omp parallel for reduction(min:min_val) reduction(max:max_val) reduction(+:sum)
```

MOST IMPORTANT LINE.

Meaning

This loop executes:

in parallel

and reduction safely combines results.

Breakdown

reduction(min:min_val)

Finds:

global minimum

Each thread computes local minimum first.
Then OpenMP combines all local minimums.

reduction(max:max_val)

Finds:

global maximum

reduction(+:sum)

Computes:

total sum

Each thread calculates partial sum.
Then:
all partial sums added together

Why Reduction Needed?

Without reduction:
Multiple threads updating same variable:
causes race condition

Example Race Condition

Suppose:
sum = 10
Thread 1 adds:
5
Thread 2 adds:
3
Simultaneously.
Incorrect updates may occur.

Reduction Solves This

Each thread gets:

private copy

Then OpenMP combines results automatically.

9. Parallel Loop

```
for (int i = 0; i < n; i++)
```

Different threads process:

different array elements simultaneously

Minimum Logic

```
if (arr[i] < min_val)
```

```
    min_val = arr[i];
```

Updates minimum.

Maximum Logic

```
if (arr[i] > max_val)
```

```
    max_val = arr[i];
```

Updates maximum.

Sum Logic

```
sum += arr[i];
```

Adds elements.

10. End Timer

```
double end = omp_get_wtime();
```

Records finish time.

Execution Time

```
(end - start)
```

Total runtime.

11. Average Calculation

```
double avg = (double)sum / n;
```

Formula:

$$\text{Average} = \frac{\text{Sum}}{n}$$

Typecasting Needed

```
(double)sum
```

ensures:

floating-point division

Example

Without typecasting:

$$5 / 2 = 2$$

(integer division)

With typecasting:

$$5 / 2 = 2.5$$

(correct average)

12. Example Execution

Input:

2 5 1 8 4

Thread Distribution Example

Thread 1:

2 5

Thread 2:

1 8

Thread 3:

4

Partial Results

Thread	Min	Max	Sum
T1	2	5	7
T2	1	8	9
T3	4	4	4

Reduction Combines

Final:

Result	Value
Min	1
Max	8
Sum	20
Avg	4

13. Time Complexity

Sequential:

$O(n)$

Parallel Reduction

Approximate:

$O\left(\frac{n}{P}\right)$

Where:

- (P) = processors/threads

Why Faster?

Because:

- array divided among multiple threads

14. Important Viva Questions

Q1. What is reduction in OpenMP?

Combining thread-local results into one final result.

Q2. Why reduction clause used?

To avoid race conditions during parallel updates.

Q3. What does reduction(+:sum) mean?

Each thread computes local sum and OpenMP adds them together.

Q4. Why INT_MAX used for minimum?

Ensures first element becomes smaller and updates minimum correctly.

Q5. Why INT_MIN used for maximum?

Ensures first element becomes larger and updates maximum correctly.

Q6. Why long long used for sum?

To avoid overflow for large values.

Q7. What is race condition?

Incorrect behavior caused by simultaneous shared variable access.

Q8. What does omp_get_wtime() do?

Measures execution time.

15. Short Practical Explanation

This program demonstrates parallel reduction operations using OpenMP. The array is processed in parallel to calculate minimum, maximum, sum, and average values. OpenMP reduction clauses safely combine thread-local results and prevent race conditions. Execution time is measured using `omp_get_wtime()`.